

HH-PIM: Dynamic Optimization of Power and Performance with Heterogeneous-Hybrid PIM for Edge AI Devices

Sangmin Jeon[†], Kangju Lee[†], Kyeongwon Lee, and Woojoo Lee^{*}

School of Intelligent Semiconductor Engineering, Chung-Ang University, Seoul, Korea
{jademin96, agl0312, since69se, space}@cau.ac.kr

Abstract—Processing-in-Memory (PIM) architectures offer promising solutions for efficiently handling AI applications in energy-constrained edge environments. While traditional PIM designs enhance performance and energy efficiency by reducing data movement between memory and processing units, they are limited in edge devices due to continuous power demands and the storage requirements of large neural network weights in SRAM and DRAM. Hybrid PIM architectures, incorporating non-volatile memories like MRAM and ReRAM, mitigate these limitations but struggle with a mismatch between fixed computing resources and dynamically changing inference workloads. To address these challenges, this study introduces a Heterogeneous-Hybrid PIM (HH-PIM) architecture, comprising high-performance MRAM-SRAM PIM modules and low-power MRAM-SRAM PIM modules. We further propose a data placement optimization algorithm that dynamically allocates data based on computational demand, maximizing energy efficiency. FPGA prototyping and power simulations with processors featuring HH-PIM and other PIM types demonstrate that the proposed HH-PIM achieves up to 60.43% average energy savings over conventional PIMs while meeting application latency requirements. These results confirm HH-PIM's suitability for adaptive, energy-efficient AI processing in edge devices.

I. INTRODUCTION

With the advent of artificial intelligence (AI), real-world applications are rapidly expanding, fueling a trend to embed AI capabilities into IoT devices across diverse fields. However, traditional server-centric data processing, such as cloud computing, faces significant energy and latency challenges due to processing and communication overloads. Consequently, distributing AI workloads to edge devices has become a promising solution, with recent research focusing on enabling on-device AI through TinyAI models that support lightweight, local computations [1], [2].

In energy-constrained edge environments, integrating Processing-in-Memory (PIM) architectures has emerged as a promising approach for executing AI applications efficiently [3]–[8]. PIM enhances performance and energy efficiency in memory-intensive tasks, such as AI applications, by minimizing the overhead of data movement between processing and memory units. Early PIM designs primarily employed volatile memories like SRAM [9] and DRAM [10], but these designs faced challenges in storing large neural network weights due to the continuous power demands of volatile memory and issues such as SRAM leakage power and the periodic refresh cycles required by DRAM. To address these limitations, PIM architectures based on non-volatile memories (NVMs), such as MRAM [11], [12] and ReRAM [13], [14], were proposed. These designs achieve high energy efficiency in weight storage, especially when combined with power-gating techniques. However, NVMs may introduce additional read/write latency, potentially impacting overall neural network performance. Consequently, recent advances in PIM design have introduced hybrid architectures combining SRAM and NVM, known as Hybrid-PIM (H-PIM). These architectures use NVM to store

weight data and SRAM as a buffer for input and output data, thereby enhancing both performance and energy efficiency [15]–[17].

However, H-PIM faces distinct limitations in achieving optimal energy efficiency during dynamic scenarios where inference loads fluctuate in real time on edge devices. For instance, an edge device running a YOLO model for real-time object detection experiences substantial variations in processing demand depending on the number of objects detected per video frame. Operating at a fixed performance level across all time intervals—typically set for peak computational load—inevitably leads to inefficient energy consumption. To address this, we observe that the mismatch between fixed computing resources and dynamically changing workloads has long been a challenge in traditional CPU-centric architectures. Established solutions, such as Dynamic Voltage and Frequency Scaling (DVFS) [18]–[21] and heterogeneous multi-processor architectures with high-performance and low-power cores [22]–[24], have been extensively researched for this purpose. Our focus centers on heterogeneous architectures, as DVFS continues to face significant challenges in edge devices due to added design complexities, such as DC-DC converters and real-time power monitoring. In contrast, heterogeneous architectures, exemplified by ARM's big.LITTLE architecture [25], are widely implemented in commercial processors, effectively improving energy efficiency by adapting to dynamic computational loads. Building on this insight, we propose configuring PIM modules, which integrate memory and Processing Elements (PEs) for independent computation, into high-performance and low-power configurations. This approach allows PIM architectures to dynamically balance performance and energy consumption throughout the application runtime.

To further elaborate this idea, we introduce Heterogeneous-Hybrid PIM (HH-PIM), an architecture designed to dynamically optimize performance and energy efficiency for AI applications on edge devices. HH-PIM integrates two distinct PIM modules: a High-Performance (HP) PIM module and a Low-Power (LP) PIM module. Each module's memory consists of a hybrid configuration of MRAM and SRAM banks, resulting in four types of memory: *HP-MRAM*, *HP-SRAM*, *LP-MRAM*, and *LP-SRAM*. Unlike conventional hybrid PIM architectures—where NVM is primarily allocated for weight storage and SRAM is reserved as an input-output buffer—HH-PIM adopts an adaptive approach. During periods of high computational demand, HH-PIM actively utilizes SRAM for weight storage as well, maximizing responsiveness to fluctuating inference loads. Additionally, to capitalize on HH-PIM's architecture, we propose an optimal data distribution algorithm that minimizes energy consumption by dynamically adjusting data allocation across the four memory types. This combinatorial optimization algorithm allocates weight data across HP-MRAM, HP-SRAM, LP-MRAM, and LP-SRAM to reduce energy use while balancing workload between HP-PIM and LP-PIM. With this design, HH-PIM efficiently adapts to the changing computational demands of AI applications, achieving significant energy savings without compromising performance.

To verify the functionality and evaluate the effectiveness of our proposed technique, we modeled the memory and PEs and performed an RTL-level design of the entire PIM processor, incorporating the HH-PIM architecture. We then conducted FPGA prototyping to confirm correct operation and measure performance of the developed PIM

This work was supported in part by Institute of Information & communications Technology Planning & Evaluation (IITP) grants funded by the Korea government (MSIT) (No. 2022-0-00971, Logic Synthesis for NVM-based PIM Computing Architecture and No. RS-2023-00277060, Development of open edge AI SoC hardware and software platform), and in part by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No. RS-2024-00345668)

[†] Sangmin Jeon and Kangju Lee contributed equally to this work.

^{*} Woojoo Lee is the corresponding author.

processor, while power measurements were obtained through synthesis using 45nm process technology and memory model simulations. Through various benchmark scenarios, we experimentally validated the energy-saving potential of the proposed HH-PIM architecture and data distribution algorithm. Results showed that our approach maximizes energy efficiency while satisfying the latency requirements of AI applications in edge computing environments. Specifically, the developed PIM processor demonstrated superior adaptability to real-time inference load variations compared to conventional PIM-based processors, achieving up to 60.43% average energy savings across different benchmark scenarios relative to the baseline processor. These outcomes validate the suitability of the HH-PIM architecture for maximizing efficiency in edge processors running AI applications.

II. HH-PIM ARCHITECTURE FOR AI EDGE PROCESSORS

Fig. 1 illustrates the proposed HH-PIM architecture, designed as a solution to dynamically maximize energy efficiency in response to changes in inference workload, while meeting the performance demands of AI applications on edge devices. Basically, HH-PIM adopts the general structure of near-memory computing architectures, as suggested in previous research [10], [26]. It comprises multiple PIM modules, each with PEs and memory banks, a controller that manages these modules, and an interface for external communication. Operating based on dedicated PIM instructions, commands received from the processor core are sequentially stored in the *PIM Instruction Queue*. Unlike conventional PIM architectures, where a single controller manages all PIM modules based on PIM instructions, HH-PIM incorporates two distinct controllers: *HP-PIM Controller* and *LP-PIM Controller*, as shown in the figure. This dual-controller setup is designed specifically to support HH-PIM's unique heterogeneous architecture, which consists of two types of clusters: HP-PIM module cluster operating at high performance with higher power consumption, and LP-PIM module cluster operating at low power with reduced performance. The HP-PIM and LP-PIM Controllers are responsible for controlling and synchronizing their respective module types.

Another notable feature of HH-PIM is its hybrid memory architecture, where the PIM module's memory consists of MRAM and SRAM with distinct characteristics and read/write latencies. The previous H-PIM architectures leverage the data characteristics of neural networks by storing large weight data in NVM to enhance energy efficiency, while using SRAM or DRAM for input/output data to improve computational performance by providing low-latency access. HH-PIM builds upon this approach, exploiting the performance and energy efficiency benefits of hybrid memory. However, to overcome the limitations of fixed memory allocation based solely on data characteristics, HH-PIM adds flexibility. When computational demand spikes and maximum PIM computing power is required, HH-PIM allows weight data to be stored in SRAM, thus enabling stable operation even in cases where traditional H-PIM architectures cannot meet the maximum latency requirements of certain applications. To realize this capability, the PIM module is designed to support variable operand counts retrieved from MRAM and SRAM during computation. Each PIM module's internal interface is controlled by the controller, which dynamically adjusts the load process based on data storage status. This design ensures synchronization of differing memory read cycles and access speeds between MRAM and SRAM in the LOAD state, guaranteeing reliable operation.

This heterogeneous and hybrid architecture of HH-PIM enables flexible adaptation to real-time variations in inference workloads for AI applications. When computational demand is high, the HP-PIM module and SRAM can be actively utilized to boost processor performance. Conversely, in low-demand scenarios, computations can be maximally allocated to the LP-PIM module to meet application latency requirements and maximize energy efficiency. To implement this strategy effectively, it is essential to design the PIM Controllers with precision, as they play a crucial role by synchronizing compo-

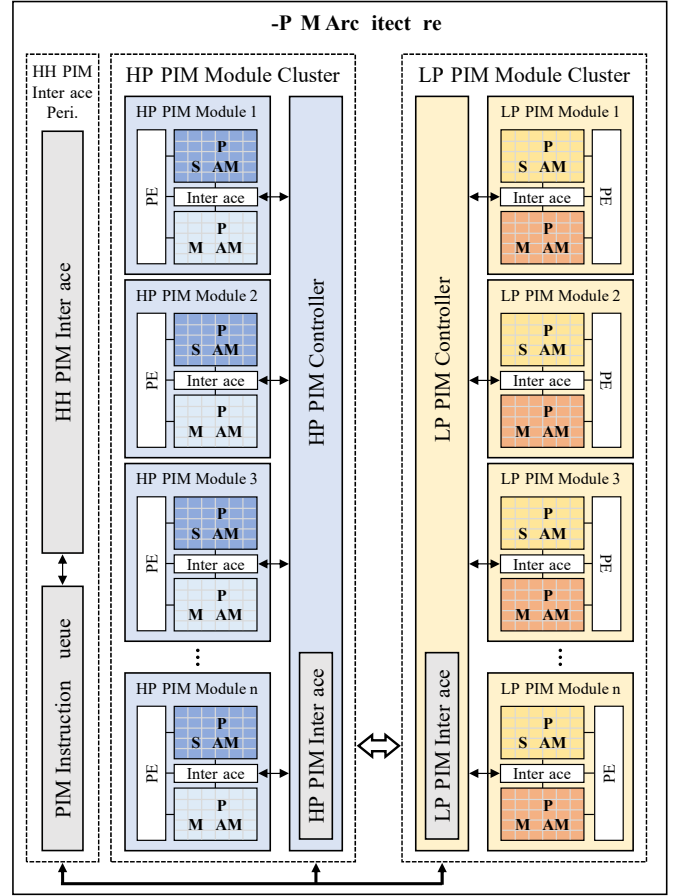


Fig. 1: Block diagram of the proposed HH-PIM architecture.

nents that operate at different speeds and minimizing data movement overhead between PIM operations.

The architecture of HP-PIM and LP-PIM Controllers is fundamentally identical. Fig. 2 shows the architecture block diagram of the HP-PIM Controller, which consists of a *State Machine*, *Instruction Decoder*, *Command Encoder*, *Data Allocator*, and *Interface Logic*. The controller operates through the basic PIM instruction cycle, which includes the FETCH-DECODE-LOAD-EXECUTE-STORE phases, managed internally by the State Machine. The Instruction Decoder decodes the fetched instruction into components such as the instruction type (*Category*), specific operation or data movement details to be executed by the PIM module (*Instruction Field*), and the target module for the operation (*Module Select Signal*). The Command Encoder then generates command signals for each PIM module based on the decoded instruction details.

To minimize overhead from frequent data movement between HP-PIM and LP-PIM modules, as well as between MRAM and SRAM within each PIM module—resulting from the dynamic energy optimization of the HH-PIM architecture—the controller design of HH-PIM incorporates a Data Allocator and separates the Interface Logic as seen in Fig. 2. The Data Allocator manages data placement to minimize external data movement during PIM operations for various computations, such as convolution and Multiply-Accumulate (MAC) operations within AI applications. Additionally, the Interface Logic is divided into a CMD Interface Logic for delivering commands to the PIM modules and a MEM Interface Logic for data movement between PIM modules. When operand data is appropriately positioned in each PIM module's memory, computations can proceed efficiently through the CMD Interface Logic alone. Furthermore, the bandwidth of the MEM Interface Logic is scaled according to the number of PIM

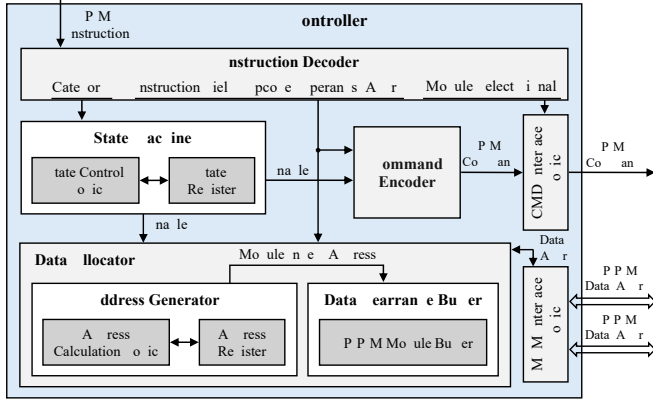


Fig. 2: Block diagram of the HP-PIM Controller architecture.

modules within each cluster, enabling parallel data transfers across clusters from multiple PIM modules simultaneously.

The controller's operation sequence for data movement between PIM modules is as follows. First, when a data placement instruction from the core prompts data transfer between HP-PIM and LP-PIM modules, the controller decodes the instruction and provides the relevant operation and address information to the Data Allocator. Based on the provided addresses, the Data Allocator accesses the memory in each PIM module via the MEM Interface Logic and stores the data to be transferred in the *Data Rearrange Buffer*. The Data Rearrange Buffer retains the data until the destination PIM module in the opposite cluster is ready for data writing, preventing data conflicts caused by the speed discrepancy between HP-PIM and LP-PIM modules. Next, the *Address Generator* within the Data Allocator generates the destination PIM module index and memory bank addresses in the opposite cluster to facilitate data movement. The generated addresses are then provided to the Data Rearrange Buffer, enabling it to place the buffered data into the memory banks of the designated PIM modules.

In summary, the PIM controllers in HH-PIM are meticulously designed to support data transfers between HP-PIM and LP-PIM modules and to effectively manage operations within each PIM module. This design maintains operational consistency and prevents data conflicts, supporting the HH-PIM architecture in achieving its design goals of real-time performance and energy efficiency optimization.

III. DYNAMIC DATA PLACEMENT STRATEGY FOR OPTIMIZING ENERGY EFFICIENCY IN HH-PIM

Optimal data placement in the HH-PIM architecture is essential for achieving maximum energy efficiency while ensuring smooth application execution without latency. In HH-PIM, each layer of a neural network is distributed across HP-PIM and LP-PIM modules for parallel computation, with the final output obtained by aggregating results from each module. When high performance is prioritized, it is critical to minimize idle times where the HP-PIM module completes its tasks quickly but must wait for the slower LP-PIM module. To achieve low output latency, a balanced distribution of data between HP-PIM and LP-PIM modules, as well as an optimal utilization ratio of MRAM and SRAM—each with different read/write latencies—must be carefully managed.

On the other hand, energy efficiency can be prioritized when AI applications do not always require the lowest possible inference latency. In such cases, HH-PIM can operate below peak performance by allocating more data to the energy-efficient LP-PIM module and increasing the usage of MRAM, thus improving overall energy efficiency while maintaining acceptable application latency. Based on this approach, we propose a dynamic data placement strategy that periodically adjusts the distribution of weight data during runtime. This strategy optimizes energy efficiency by shifting computational

loads according to real-time demands, focusing on maximizing energy savings while meeting the minimum latency requirements of the application.

A. Problem Definition

In the proposed weight data placement strategy, data redistribution occurs every predefined interval T , referred to as the *time slice*. We define a set of PIM operations generated by a single inference process in an AI application as a *task*. These tasks, generated by inference, are stored in a task buffer, and HH-PIM sequentially processes the tasks stored from the previous *time slice* within the current *time slice*. This approach ensures that the operational latency of HH-PIM does not exceed $2T$, thereby maintaining the inference latency required by AI applications. In this framework, the number of tasks stored in the task buffer determines the minimum time $t_{constraint}$ needed for each task to be processed within the *time slice* without delay. Specifically, each task's processing time t_{task} must not exceed $t_{constraint}$ to guarantee that all tasks are completed within the allotted *time slice*, ensuring the system adheres to the required maximum latency.

The proposed method aims to achieve the most energy-efficient data distribution for each task within the time constraint $t_{constraint}$. The HH-PIM architecture includes four distinct storage spaces: HP-MRAM, HP-SRAM, LP-MRAM, and LP-SRAM, each with different power consumption and latency characteristics. Consequently, we are faced with a discrete combinatorial optimization problem of determining the optimal placement of each weight data in order to minimize energy consumption. This problem can be reduced to a well-known *knapsack problem* [27], with necessary adaptations for our specific context. Here, each storage space corresponds to an item i in the knapsack problem, and the time constraint $t_{constraint}$, task execution time t_{task} , and energy consumption per task E_{task} represent the knapsack's capacity, item weight, and item value, respectively. However, this problem is more complex than the standard knapsack problem, as it exhibits the characteristics of both an unbounded knapsack (due to the multiple selections of a particular storage space) and a multi-choice knapsack (due to the fixed total number of selected storage spaces, equivalent to the number of weight data).

The mathematical formulation of this problem is as follows:

$$\begin{aligned} \text{Minimize } E_{task} &= \sum_{i=1}^n e_i \cdot x_i \\ \text{subject to: } &\sum_{i=1}^n t_i \cdot x_i \leq t_{constraint}, \quad \sum_{i=1}^n x_i = k, \\ &x_i \in \mathbb{Z}_{\geq 0}, \quad \forall i = 1, 2, \dots, n. \end{aligned} \quad (1)$$

where n denotes the number of storage spaces, x_i represents the number of data assigned to storage space i , and k is the total number of weight data to be stored. Additionally, t_i and e_i denote the computation time per weight and the energy consumption per weight in storage space i , respectively. This formula defines the objective function to minimize energy consumption per task, E_{task} , along with constraints on execution time and the total count of selected items.

B. Proposed Solution for Optimized Data Placement

Since this problem is inherently more complex than the standard knapsack problem, it is naturally NP-hard. To address the optimization problem defined in (1), we propose a bottom-up dynamic programming (DP) approach. This method uses a DP table to store intermediate results, avoiding redundant calculations and solving the problem incrementally. The recurrence relation is defined as follows:

$$dp[i][t][k] = \begin{cases} dp[i-1][t][k], & (t_i \cdot k > t) \\ \min(dp[i-1][t][k], \\ dp[i][t-t_i][k-1] + e_i), & (t_i \cdot k \leq t) \end{cases} \quad (2)$$

Algorithm 1 Finding Optimal Data Placement with DP.

```

1: function KNAPSACK_MIN_ENERGY
2:   Set  $dp[i][t][k] = \infty$  and  $count[i][t][k] = 0$  for all  $i, t$  and  $k$ .
3:   Set  $dp[i][t][0] = 0$  for all  $i$  and  $t$ .
4:   for  $i$  from 1 to  $n/2$ :
5:     for  $k$  from 1 to  $K$ :
6:       for  $t$  from 1 to  $T$ :
7:         if  $t_i \leq t$  then
8:           Calculate the possible previous state time index  $t-t_i$ 
9:            $dp[i][t][k] = \min(dp[i-1][t-t_i][k], dp[i][t-t_i][k-1] + e_i)$ 
10:          Update  $count[i][t][k]$  based on the chosen minimum path.
11:         else
12:            $dp[i][t][k] = dp[i-1][t][k]$ 
13:            $count[i][t][k] = count[i-1][t][k]$ 
14:         end if
15:       end for
16:     end for
17:   end for
18: end function

```

where i, t , and k denote the current storage space being considered, the time constraint, and the number of weights, respectively. The value $dp[i][t][k]$ represents the minimum energy consumption required to store exactly k weight data across the first i storage spaces while satisfying the time constraint t . In this equation, when $t_i \cdot k > t$, storing k weights in the current storage space exceeds the time constraint. Therefore, in this case, the value from the previous storage space $dp[i-1][t][k]$ is carried forward. Conversely, if $t_i \cdot k \leq t$, the recurrence considers the minimum between $dp[i-1][t][k]$ and $dp[i][t-t_i][k-1] + e_i$, where the latter term represents the energy consumption if one additional weight is stored in the current storage space i , based on the previous results.

Meanwhile, in implementing the solution based on (2), it is essential to consider the architectural constraints of HH-PIM. HH-PIM allows for parallel execution of weight data distributed between HP-PIM and LP-PIM clusters; however, parallel processing is not feasible for weights stored across MRAM and SRAM within each module. To address this, we propose a partitioned approach where the DP algorithm is applied separately to HP-PIM and LP-PIM clusters, generating individual DP tables for each cluster. By combining these tables, we identify the optimal allocation of weights that minimizes energy consumption within the time constraint, yielding the best combination of k values, (k_{hp}, k_{lp}) , where k_{hp} and k_{lp} represent the number of weights assigned to HP-PIM and LP-PIM clusters, respectively. This approach effectively accounts for both inter-cluster parallelism and intra-module serialization, optimizing energy efficiency while adhering to the operational constraints of HH-PIM.

First, Algorithm 1 presents the pseudo-code of the algorithm based on (2). Here, K denotes the total number of weight data to be stored in HH-PIM, and $count[i][t][k]$ is a variable used to trace the path to the optimal energy state. Lines 2 and 3 set the base conditions for the recurrence relation, while lines 7-13 correspond to the recurrence relation defined in (2). Notably, because Algorithm 1 is performed separately for both HP-PIM and LP-PIM, the iteration for i in line 4 only proceeds up to $n/2$.

Next, Algorithm 2 finds the optimal combination of k values that minimizes energy consumption using the results of the DP tables dp_{hp} and dp_{lp} generated for HP-PIM and LP-PIM, respectively. Here, min_energy denotes the minimum energy consumption, while k_{opt_hp} and k_{opt_lp} represent the number of weights assigned to HP-PIM and LP-PIM at this minimum. As a result, Algorithm 2 yields $allocation_state[n][t]$, representing the distribution state of weight data for each time constraint. In the proposed method, both Algorithm 1 and Algorithm 2 are performed only once during the application initialization phase to construct a Look-up Table (LUT) for the final output, $allocation_state$. This LUT allows rapid determination of the optimal weight placement state for varying $t_{constraint}$

Algorithm 2 Finding Optimal (k_{hp}, k_{lp}) .

```

1: function SET_ALLOCATION_STATE
2:   for  $t$  from 0 to  $T$ :
3:     Set  $min\_energy = \infty$ ,  $k_{opt\_hp} = 0$ ,  $k_{opt\_lp} = 0$ 
4:     for  $k_{hp}$  from 1 to  $K$ :
5:        $k_{lp} = K - k_{hp}$ 
6:       if  $dp_{hp}[n/2][t][k_{hp}] + dp_{lp}[n/2][t][k_{lp}] < \infty$  then
7:         if  $min\_energy > dp_{hp}[n/2][t][k_{hp}] + dp_{lp}[n/2][t][k_{lp}]$  then
8:            $min\_energy = dp_{hp}[n/2][t][k_{hp}] + dp_{lp}[n/2][t][k_{lp}]$ 
9:            $k_{opt\_lp} = k_{lp}$ 
10:        end if
11:      end if
12:      if  $min\_energy$  remains  $\infty$ , continue to next  $t$ 
13:       $k_{opt\_hp} = K - k_{opt\_lp}$ 
14:      Set  $allocation\_state[n][t]$  with current  $k_{opt\_hp}$  and  $k_{opt\_lp}$ 
15:    end for
16:  end for
17: end function

```

TABLE I: Developed specifications for HH-PIM and other PIM architectures.

Architecture	PIM Module Configuration	Memory Types (per module)
Baseline-PIM	8 HP-PIM	128kB SRAM
Heterogeneous-PIM	4 HP-PIM + 4 LP-PIM	128kB SRAM
Hybrid-PIM	8 HP-PIM	64kB MRAM + 64kB SRAM
HH-PIM	4 HP-PIM + 4 LP-PIM	64kB MRAM + 64kB SRAM

values required at each *time slice* during application runtime. Furthermore, the calculation of $t_{constraint}$ at runtime incorporates the data movement overhead time needed for transitioning from the previous *time slice* data allocation to the new one, ensuring no inference delay arises due to data movement overhead.

The time complexities of Algorithm 1 and Algorithm 2 are $O(n \cdot T \cdot K)$ and $O(T \cdot K)$, respectively. Although these algorithms are executed only once during application initialization, the full *time slice* range for T and the large number of weight data K , often tens to hundreds of thousands in lightweight neural network models, may impose considerable overhead for energy optimization across all points in an edge processor. To mitigate this, we limit the resolution of optimization to ensure that the total computation time does not exceed 1% of each *time slice*, thereby avoiding excessive fine-grained calculations. The resulting optimal data distribution and improvements in energy efficiency are presented in the following section, along with detailed experimental settings.

IV. EXPERIMENTAL WORK

A. Development and Evaluation of PIM Processors for Power and Performance Assessment

To evaluate the effectiveness of the proposed HH-PIM architecture and its data placement optimization strategy, we first implemented the HH-PIM architecture from Fig. 1 at the RTL level. The HP and LP clusters were each configured with four HP-PIM and four LP-PIM modules, with each PIM module equipped with 64kB of MRAM and SRAM. For a fair and precise comparison, we established Baseline-PIM, Heterogeneous-PIM (Hetero.-PIM), and Hybrid-PIM as comparison groups, each also implemented at the RTL level. Table I presents the specifications of each comparison group.

Subsequently, we developed processors equipped with the proposed HH-PIM and each of the comparative PIM architectures. RTL design and simulation were conducted using RISC-V eXpress (RVX) [28], [29]. The diagram on the right side of Fig. 3 illustrates the architecture of the processor equipped with HH-PIM, which is built with a single RISC-V Rocket [30] core. To facilitate efficient transmission and reception of large-scale AI application data, the HH-PIM communicates with the core through the AXI protocol, offering high bandwidth and

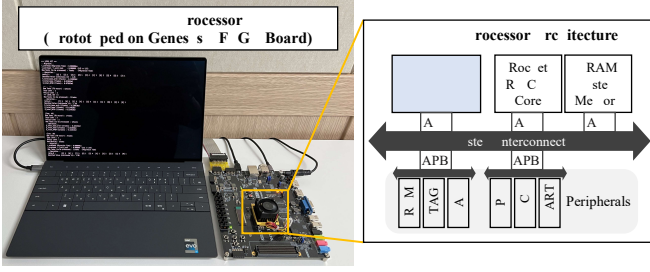


Fig. 3: Design and FPGA prototyping of the HH-PIM processor.

TABLE II: FPGA prototype resource utilization.

IPs	LUTs	FFs	BRAMs	DSPs
RISC-V Rocket Core	14,998	9,762	12	4
Peripherals	4,704	7,159	-	-
System Interconnect	5,237	7,720	-	-
HP-PIM Module	968	1,055	32	2
HP-PIM Module Controller	2,823	875	-	-
Total (HP-PIM module cluster)	6,951	5,460	128	8
LP-PIM Module	1,074	1,094	32	2
LP-PIM Module Controller	2,149	875	-	-
Total (LP-PIM module cluster)	6,680	5,616	128	8

low latency. For the system interconnect, we utilized μNoC [31], a lightweight Network-on-Chip optimized for edge devices.

Next, we performed functional verification and performance evaluation through FPGA prototyping of the developed processors. First, we modeled the operation and latency of SRAM at the RTL level, with Table III reporting the read/write latencies used in the model. These results were obtained using the NVSim simulation tool [32] at a 45nm process, with different operating voltages of 1.2V for HP-PIM and 0.8V for LP-PIM. The 0.8V operating voltage of LP-MRAM, in particular, is based on recent specifications of fabricated STT-MRAM chips [11], [33]. We then programmed the developed processors on a Genesys2 Kintex-7 FPGA board [34]. The operating clock frequency of the prototypes was set to 50 MHz, and memory latencies were scaled according to Table III. The photograph on the right side of Fig. 3 shows the prototype of the HH-PIM processor, while Table II reports the resource consumption of the prototype.

We verified the correct operation of the FPGA prototypes and measured performance metrics by running benchmark applications, expecting similar results to those observed on an SoC operating at 50 MHz [28]. For the AI benchmark applications, we used TinyML models based on CNN deep learning backbones such as EfficientNet-B0 [35], MobileNetV2 [36], and ResNet-18 [37], which are suitable for edge AI devices. We extracted the characteristics and operations of these models, calculated the proportion of PIM operations relative to the total operations, and incorporated these ratios into the benchmark applications. The *time slice* for performing the data placement algorithm in each TinyML model was set to allow up to 10 inferences per *time slice*, representing the scenario in which HH-PIM operates at maximum performance. Table IV reports the characteristics of the models incorporated into the benchmark applications. Additionally, to reflect dynamic computational load variations encountered at runtime, we configured six workload scenarios as shown in Fig. 4. These scenarios include a consistently low workload pattern (Case 1), a consistently high workload pattern (Case 2), periodic spike patterns (Case 3 and 4), a pulsing pattern with alternating high and low workloads (Case 5), and a random workload pattern (Case 6).

TABLE III: Latency comparison of HP-PIM and LP-PIM modules.

Latency (ns)	STT-MRAM		SRAM		PE
	Read	Write	Read	Write	
HP-PIM ($V_{dd} = 1.2V$)	2.62	11.81	1.12	1.12	5.52
LP-PIM ($V_{dd} = 0.8V$)	2.96	14.65	1.41	1.41	10.68

TABLE IV: TinyML model specs and PIM operation ratios.

Model	# Param	# MAC	PIM Operation	
EfficientNet-B0	95k	3.245M	85%	INT8 Quantized & Pruned
MobileNetV2	101k	2.528M	80%	
ResNet-18	256k	29.580M	75%	

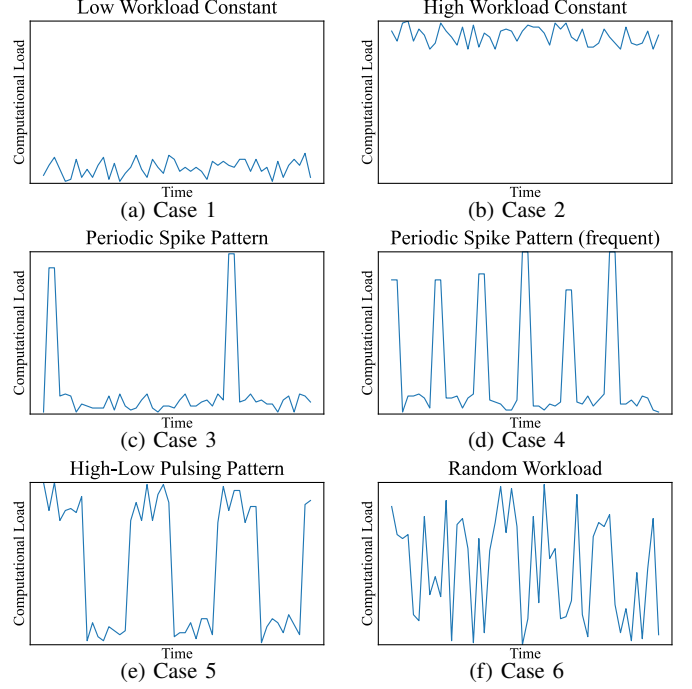


Fig. 4: Various workload scenarios of the AI benchmark app.

TABLE V: Power consumption (mW) across memory types in HP-PIM (1.2V) and LP-PIM (0.8V).

Power	STT-MRAM		SRAM		PE	
	Dynamic (Read/Write)	Static	Dynamic (Read/Write)	Static	Dynamic	Static
HP-PIM	428.48 / 133.78	2.98	508.93 / 500	23.29	0.9	0.48
LP-PIM	179.05 / 47.78	0.84	177.3 / 177.3	5.45	0.51	0.25

workloads (Case 5), and a random workload pattern (Case 6). The spike and pulse patterns simulate realistic scenarios in AI applications on edge devices, where computational demands periodically surge.

To measure the power consumption of the developed processors, we then synthesized them at a 45nm process technology. The RTL design, excluding memory, was synthesized using Synopsys Design Compiler with the 45nm Nangate PDK [38], while the memory power values were obtained through NVSim simulations at 45nm technology, as previously described. Table V details the power consumption of each memory type. These values were subsequently combined with benchmark-specific execution times, as measured from FPGA prototypes, to calculate total energy consumption. A detailed analysis of energy consumption, along with energy-saving results across various benchmark scenarios shown in Fig. 5, is discussed further in the following section.

B. Comprehensive Analysis of Energy Savings

Fig. 6 reports the results of applying the proposed data placement optimization algorithm to benchmark applications executed on HH-PIM. The x-axis represents the $t_{constraint}$ within a *time slice* (T), while the left y-axis indicates the memory utilization determined by the optimal data placement under the given $t_{constraint}$, and the right y-axis represents the corresponding energy consumption

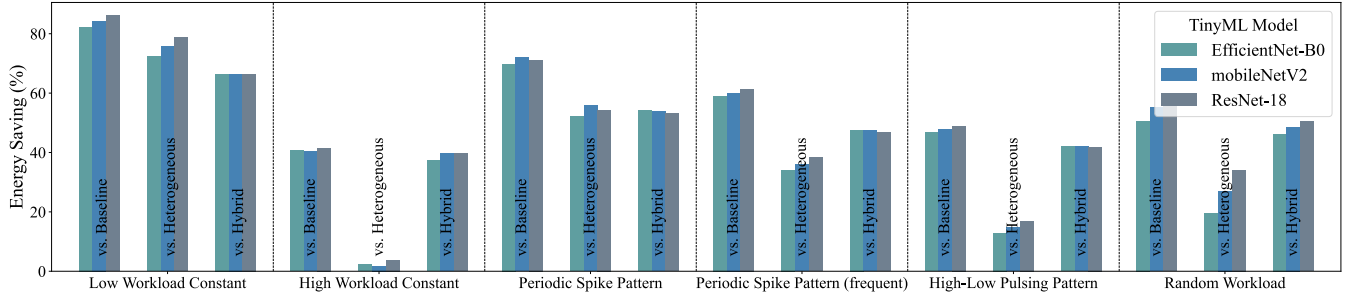


Fig. 5: Energy savings of the HH-PIM over the Baseline-, Heterogeneous-, and Hybrid-PIM across benchmark scenarios.

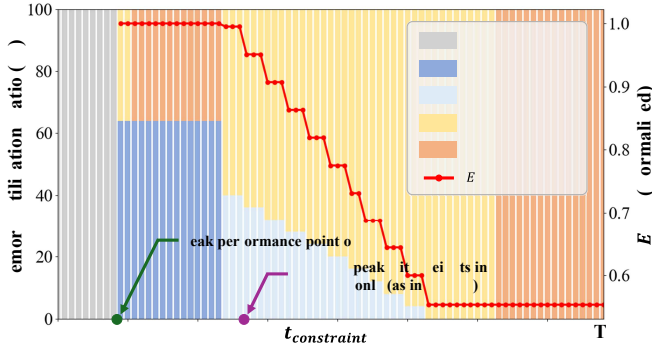


Fig. 6: Memory utilization and energy consumption across $t_{\text{constraint}}$ within time slice (T) based on HH-PIM's optimized data placement.

E_{task} of HH-PIM. The gray region in the graph denotes cases where $t_{\text{constraint}}$ is too small, i.e., the required performance is too high for HH-PIM to handle. The green dot marks the point where HH-PIM operates at peak performance, satisfying the application's performance requirements. At this point, HH-PIM actively utilizes SRAMs for data storage, with the total neural network data stored in a 16:9 ratio between HP-SRAM and LP-SRAM, minimizing PIM module idle times. The inference times at this point are measured as 31.06ms, 25.71ms, and 320.87ms for the EfficientNet-B0, MobileNetV2, and ResNet-18 benchmarks, respectively. As expected, energy consumption peaks at this point, and E_{task} is normalized to this value in the graph. Additionally, the purple dot indicates the peak performance point of HH-PIM when weights are stored only in MRAM (as in previous H-PIMs). At this point, the inference times for the EfficientNet-B0, MobileNetV2, and ResNet-18 benchmarks are 44.5ms, 36.84ms, and 459.74ms, respectively. This demonstrates that utilizing both SRAM and MRAM for weight storage, as proposed, outperforms the traditional method in terms of performance.

Fig. 6 demonstrates the sequential allocation of weight data across memory types as $t_{\text{constraint}}$ increases. Specifically, the distribution progresses through combinations of HP-SRAM and LP-MRAM, HP-MRAM and LP-SRAM, LP-SRAM alone, and finally LP-MRAM alone. This sequence reflects the balance between performance and energy efficiency across memory types. In the range where $t_{\text{constraint}}$ is longest, all weight data are stored exclusively in LP-MRAM, which operates at minimal power. In this configuration, other memory types are deactivated through power-gating, eliminating standby power and maximizing the processor's energy efficiency. Consequently, in this highly efficient region, HH-PIM achieves up to a 43.17% reduction in E_{task} compared to unoptimized data allocation. This significant reduction underscores the practical value of the proposed algorithm in real-world edge AI applications. Additionally, in the mid-range of $t_{\text{constraint}}$, E_{task} exhibits a quasi-linear decline with intermittent plateaus as $t_{\text{constraint}}$ increases. This pattern reflects the algorithm's capability to shift data progressively to lower-

TABLE VI: Energy Savings (ES) by HH-PIM for Cases 3–6.

	ES (%)	Baseline-PIM	Hetero-PIM	H-PIM
Case 3: Periodic Spike	over	72.01	55.78	54.09
Case 4: Periodic Spike (frequent)		61.46	38.38	47.60
Case 5: High-Low Pulsing		48.94	16.89	42.10
Case 6: Random		59.28	34.14	50.52

power memory while meeting inference latency constraints. These results demonstrate that the proposed solution achieves optimal energy efficiency within the constraints of application inference latency for each $t_{\text{constraint}}$, ensuring that HH-PIM can dynamically allocate data to balance energy savings and performance effectively.

Fig. 5 presents the energy savings achieved by HH-PIM compared to Baseline-PIM, Hetero.-PIM, and H-PIM (Hybrid-PIM) when executing the benchmark application across 50 time slices under various inference pattern scenarios from Fig. 4. The best-case scenario for HH-PIM is Case 1 (low workload constant), where the inference pattern remains low across all intervals. In this scenario, HH-PIM achieved energy savings of up to 86.23%, 78.7%, and 66.5% compared to Baseline-, Hetero.-, and H-PIM, respectively. Conversely, the worst-case scenario occurs in Case 2 (high workload constant), where the inference pattern remains consistently high. Here, HH-PIM still delivered notable energy savings of up to 41.46% and 39.69% compared to Baseline- and H-PIM, respectively. However, savings against Hetero.-PIM were limited to 3.72%, as both HH-PIM and Hetero.-PIM primarily utilize HP-SRAM and LP-SRAM in this scenario, resulting in minimal differences. Detailed energy savings for the other cases are reported in Table VI. HH-PIM achieved the highest energy savings over the baseline in ResNet-18, with up to 8.59% difference across cases. On average, HH-PIM achieved energy savings of up to 60.43%, 36.3%, and 48.58% compared to Baseline-PIM, Hetero.-PIM, and H-PIM, respectively.

V. CONCLUSION

In this study, we proposed the HH-PIM architecture designed to efficiently execute AI applications on edge devices. The architecture integrates HP MRAM-SRAM PIM modules and LP MRAM-SRAM PIM modules to achieve a balance between performance and energy efficiency. Additionally, we introduced a data placement optimization algorithm that dynamically allocates data based on computational demand, maximizing energy savings while maintaining application performance requirements. To validate the effectiveness of the proposed architecture and algorithm, we designed processors incorporating HH-PIM and established comparison baselines with processors equipped with Baseline-, Heterogeneous-, and Hybrid-PIM architectures. These designs were evaluated through FPGA prototyping and power simulations. By running various edge AI application benchmarks, the proposed HH-PIM demonstrated its superiority, achieving up to 60.43% average energy savings compared to conventional PIM architectures, while meeting application performance requirements.

REFERENCES

- [1] M. Shafique, T. Theodoridis, V. J. Reddy, and B. Murmann, "TinyML: Current progress, research challenges, and future roadmap," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2021, pp. 1303–1306.
- [2] Y. Dong, T. Jia, K. Du, Y. Jing, Q. Wang, P. Zhan, Y. Zhang, F. Yan, Y. Ma, Y. Liang *et al.*, "A model-specific end-to-end design methodology for resource-constrained TinyML hardware," in *ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2023, pp. 1–6.
- [3] Q. Zheng, S. Li, Y. Wang, Z. Li, Y. Chen, and H. H. Li, "Accelerating sparse attention with a reconfigurable non-volatile processing-in-memory architecture," in *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2023, pp. 1–6.
- [4] X. Yang, K. Zhu, X. Tang, M. Wang, M. Zhan, N. Lu, J. P. Kulkarni, D. Z. Pan, Y. Liu, and N. Sun, "An in-memory-computing charge-domain ternary CNN classifier," *IEEE Journal of Solid-State Circuits*, vol. 58, no. 5, pp. 1450–1461, 2023.
- [5] Y.-D. Chih, P.-H. Lee, H. Fujiwara, Y.-C. Shih, C.-F. Lee, R. Naous, Y.-L. Chen, C.-P. Lo, C.-H. Lu, H. Mori *et al.*, "16.4 an 89TOPS/W and 16.3 TOPS/mm² all-digital SRAM-based full-precision compute-in-memory macro in 22nm for machine-learning edge applications," in *IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 64. IEEE, 2021, pp. 252–254.
- [6] J. Heo, J. Kim, S. Lim, W. Han, and J.-Y. Kim, "T-PIM: An energy-efficient processing-in-memory accelerator for end-to-end on-device training," *IEEE Journal of Solid-State Circuits*, vol. 58, no. 3, pp. 600–613, 2022.
- [7] K. Park, H. Jeong, S. Kim, J. Shin, M. Kim, and K. J. Lee, "A 701.7 TOPS/W Compute-in-Memory processor with time-domain computing for spiking neural network," *IEEE Transactions on Circuits and Systems I: Regular Papers*, pp. 1–11, 2024.
- [8] K. Lee, S. Jeon, K. Lee, W. Lee, and M. Pedram, "Radar-PIM: Developing IoT processors utilizing Processing-in-Memory architecture for ultra-wideband radar-based respiration detection," *IEEE Internet of Things Journal*, 2024.
- [9] J.-W. Su, X. Si, Y.-C. Chou, T.-W. Chang, W.-H. Huang, Y.-N. Tu, R. Liu, P.-J. Lu, T.-W. Liu, J.-H. Wang *et al.*, "15.2 A 28nm 64Kb inference-training two-way transpose multibit 6T SRAM compute-in-memory macro for AI edge chips," in *IEEE International Solid-State Circuits Conference (ISSCC)*. IEEE, 2020, pp. 240–242.
- [10] S. Lee, S.-h. Kang, J. Lee, H. Kim, E. Lee, S. Seo, H. Yoon, S. Lee, K. Lim, H. Shin *et al.*, "Hardware architecture and software stack for PIM based on commercial DRAM technology: Industrial product," in *ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 43–56.
- [11] Y.-C. Chiu, W.-S. Khwa, C.-Y. Li, F.-L. Hsieh, Y.-A. Chien, G.-Y. Lin, P.-J. Chen, T.-H. Pan, D.-Q. You, F.-Y. Chen *et al.*, "A 22nm 8Mb STT-MRAM near-memory-computing macro with 8b-precision and 46.4–160.1 TOPS/W for edge-ai devices," in *2023 IEEE International Solid-State Circuits Conference (ISSCC)*. IEEE, 2023, pp. 496–498.
- [12] H. Cai, Z. Bian, Y. Hou, Y. Zhou, Y. Guo, X. Tian, B. Liu, X. Si, Z. Wang, J. Yang *et al.*, "33.4 A 28nm 2Mb STT-MRAM computing-in-memory macro with a refined bit-cell and 22.4–41.5 TOPS/W for ai inference," in *2023 IEEE International Solid-State Circuits Conference (ISSCC)*. IEEE, 2023, pp. 500–502.
- [13] C.-X. Xue, Y.-C. Chiu, T.-W. Liu, T.-Y. Huang, J.-S. Liu, T.-W. Chang, H.-Y. Kao, J.-H. Wang, S.-Y. Wei, C.-Y. Lee *et al.*, "A CMOS-integrated compute-in-memory macro based on resistive random-access memory for AI edge devices," *Nature Electronics*, vol. 4, no. 1, pp. 81–90, 2021.
- [14] T. Yang, D. Li, Y. Han, Y. Zhao, F. Liu, X. Liang, Z. He, and L. Jiang, "PIMGCN: A ReRAM-based PIM design for graph convolutional network acceleration," in *ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2021, pp. 583–588.
- [15] Y. Zhu, Z. Zhu, G. Dai, F. Tu, H. Sun, K.-T. Cheng, H. Yang, and Y. Wang, "Pim-hls: An automatic hardware generation tool for heterogeneous processing-in-memory-based neural network accelerators," in *ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2023, pp. 1–6.
- [16] H. Kim, H. Ye, T. Mudge, R. Dreslinski, and N. Talati, "RecPIM: A PIM-enabled DRAM-RRAM hybrid memory system for recommendation models," in *IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*. IEEE, 2023, pp. 1–6.
- [17] F. Zhang, A. Sridharan, W. Tsai, Y. Chen, S. X. Wang, and D. Fan, "Efficient memory integration: MRAM-SRAM hybrid accelerator for sparse on-device learning," in *ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.
- [18] W. Lee, Y. Wang, and M. Pedram, "VRCon: Dynamic reconfiguration of voltage regulators in a multicore platform," in *Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2014, pp. 1–6.
- [19] S. K. Panda, M. Lin, and T. Zhou, "Energy-efficient computation offloading with DVFS using deep reinforcement learning for time-critical iot applications in edge computing," *IEEE Internet of Things Journal*, vol. 10, no. 8, pp. 6611–6621, 2022.
- [20] W. Lee, Y. Wang, and M. Pedram, "Optimizing a reconfigurable power distribution network in a multicore platform," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 34, no. 7, pp. 1110–1123, 2015.
- [21] H. Bouzidi, M. Odema, H. Ouarnoughi, M. A. Al Faruque, and S. Niar, "HADAS: Hardware-aware dynamic neural architecture search for edge performance scaling," in *Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2023, pp. 1–6.
- [22] K. Lee, K. Bong, C. Kim, J. Park, and H.-J. Yoo, "An energy-efficient parallel multi-core ADAS processor with robust visual attention and workload-prediction DVFS for real-time HD stereo stream," in *IEEE Symposium in Low-Power and High-Speed Chips (COOL CHIPS XIX)*, 2016, pp. 1–3.
- [23] J. Park, E. Choi, K. Lee, J.-J. Lee, K. Han, and W. Lee, "Developing an ultra-low power RISC-V processor for anomaly detection," in *Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2023, pp. 1–2.
- [24] E. Choi, J. Park, K. Lee, J.-J. Lee, K. Han, and W. Lee, "Day-Night architecture: Development of an ultra-low power RISC-V processor for wearable anomaly detection," *Journal of Systems Architecture*, vol. 152, p. 103161, 2024.
- [25] ARM, <https://www.arm.com/technologies/big-little>, Accessed 19 11. 2024.
- [26] J. H. Kim, S.-H. Kang, S. Lee, H. Kim, Y. Ro, S. Lee, D. Wang, J. Choi, J. So, Y. Cho *et al.*, "Aquabolt-XL HBM2-PIM, LPDDR5-PIM with in-memory processing, and AXDIMM with acceleration buffer," *IEEE Micro*, vol. 42, no. 3, pp. 20–30, 2022.
- [27] H. Kellerer, U. Pferschy, and D. Pisinger, *Knapsack Problems*. Springer, 2004.
- [28] K. Han, S. Lee, K.-I. Oh, Y. Bae, H. Jang, J.-J. Lee, W. Lee, and M. Pedram, "Developing TEI-aware ultralow-power SoC platforms for IoT end nodes," *IEEE Internet of Things Journal*, vol. 8, no. 6, pp. 4642–4656, 2020.
- [29] H. Jang, K. Han, S. Lee, J.-J. Lee, S.-Y. Lee, J.-H. Lee, and W. Lee, "Developing a multicore platform utilizing open RISC-V cores," *IEEE Access*, vol. 9, pp. 120 010–120 023, 2021.
- [30] SiFIVE, <https://github.com/chipsalliance/rocket-chip>, Accessed 19 11. 2024.
- [31] K. Han, S. Lee, J.-J. Lee, W. Lee, and M. Pedram, "TIP: A temperature effect inversion-aware ultra-low power system-on-chip platform," in *IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*, 2019, pp. 1–6.
- [32] X. Dong, C. Xu, Y. Xie, and N. P. Jouppi, "Nvsm: A circuit-level performance, energy, and area model for emerging nonvolatile memory," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 31, no. 7, pp. 994–1007, 2012.
- [33] P.-H. Lee, C.-F. Lee, Y.-C. Shih, H.-J. Lin, Y.-A. Chang, C.-H. Lu, Y.-L. Chen, C.-P. Lo, C.-C. Chen, C.-H. Kuo *et al.*, "33.1 A 16nm 32Mb embedded STT-MRAM with a 6ns read-access time, a 1M-cycle write endurance, 20-year retention at 150° c and MTJ-OTP solutions for magnetic immunity," in *IEEE International Solid-State Circuits Conference (ISSCC)*. IEEE, 2023, pp. 494–496.
- [34] Genesys2, <https://diligent.com/shop/genesys-2-kintex-7-fpga-development-board>, Accessed 19 11. 2024.
- [35] M. Tan and Q. Le, "Efficientnet: Rethinking model scaling for convolutional neural networks," in *International conference on machine learning*. PMLR, 2019, pp. 6105–6114.
- [36] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 4510–4520.
- [37] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [38] NCSU, <https://eda.ncsu.edu/freepdk/freepdk45>, Accessed 19 11. 2024.